

Bioinfo Assignment 3

Bohan YANG
Student ID: 2330016056

December 7, 2025

Question 1

Let $P = (p_{xy})_{x,y \in \{A,C,G,T\}}$ be the given transition matrix.

The root is

$$X_0 = AAA,$$

so at each site $k = 1, 2, 3$ the root state is

$$R_k = A.$$

The observed sequences are

$$X_1 = AGT, \quad X_2 = AAC, \quad X_3 = AGG.$$

Because sites evolve i.i.d., the likelihood of a tree topology T is

$$L(T) = \prod_{k=1}^3 L_k(T), \quad \ell(T) = \log L(T) = \sum_{k=1}^3 \log L_k(T),$$

where $L_k(T)$ is the likelihood for site k .

Let $H_k \in \{A, C, G, T\}$ be the (unobserved) state at the internal node h at site k . Using the tree structures, the per-site likelihoods are:

Tree 1: $0 \rightarrow h, h \rightarrow 1, h \rightarrow 2, 0 \rightarrow 3$

$$L_k(T_1) = p_{A,X_{3k}} \sum_h p_{A,h} p_{h,X_{1k}} p_{h,X_{2k}}.$$

Tree 2: $0 \rightarrow h, h \rightarrow 3, h \rightarrow 2, 0 \rightarrow 1$

$$L_k(T_2) = p_{A,X_{1k}} \sum_h p_{A,h} p_{h,X_{3k}} p_{h,X_{2k}}.$$

Tree 3: $0 \rightarrow h, h \rightarrow 1, h \rightarrow 3, 0 \rightarrow 2$

$$L_k(T_3) = p_{A,X_{2k}} \sum_h p_{A,h} p_{h,X_{1k}} p_{h,X_{3k}}.$$

Here X_{ik} is the nucleotide at site k in sequence X_i , and p_{xy} is the entry of P for parent base x and child base y . The three site patterns (including the root) are

k	R_k	X_{1k}	X_{2k}	X_{3k}
1	A	A	A	A
2	A	G	A	G
3	A	T	C	G

Substituting these bases and the entries of P into the formulas above gives the following per-site likelihoods (numerical values):

	L_1	L_2	L_3
T_1	0.9074011636	4.6821327×10^{-5}	1.7933930×10^{-6}
T_2	0.9074011636	4.6821327×10^{-5}	1.3051850×10^{-6}
T_3	0.9074011636	6.5951599×10^{-3}	1.3056000×10^{-6}

Therefore,

$$\begin{aligned} L(T_1) &= L_1 L_2 L_3 \approx 7.62 \times 10^{-11}, & \ell(T_1) &\approx -23.30, \\ L(T_2) &\approx 5.55 \times 10^{-11}, & \ell(T_2) &\approx -23.62, \\ L(T_3) &\approx 7.81 \times 10^{-9}, & \ell(T_3) &\approx -18.67. \end{aligned}$$

Since T_3 has the largest (least negative) log-likelihood, the maximum-likelihood phylogenetic tree is

Tree 3: $0 \rightarrow h, h \rightarrow 1, h \rightarrow 3, 0 \rightarrow 2$.

Question 2

1. Methodology

Data processing and analysis were conducted using Python (Scanpy and GSEAPy packages).

- **Clustering:** Cells were normalized, log-transformed, and clustered using the Leiden algorithm (*resolution* = 0.5).
- **Differential Expression:** The ****Wilcoxon rank-sum test**** was applied to compare the two largest clusters. Significant genes were filtered based on $P_{adj} < 0.05$ and $FoldChange > 2$.
- **Enrichment:** Functional analysis was performed using the GO Biological Process database (Mouse).

2. Results

The two largest clusters identified were Cluster 0 (687 cells) and Cluster 1 (496 cells).

A total of 366 differentially expressed genes were detected. The top upregulated genes in Cluster 0 include *NRL*, *NR2E3*, and *RS1* (see Table 1).

Gene	Adj. P-value	Fold Change
NRL	1.02×10^{-141}	6.69
NR2E3	1.18×10^{-140}	6.49
RS1	3.57×10^{-138}	6.37
...	...	...

Table 1: Top differentially expressed genes between Cluster 0 and Cluster 1.

The gene set enrichment analysis indicated that the most relevant biological function is **visual perception (GO:0007601)**. This aligns with the presence of photoreceptor-specific markers (e.g., *NRL*) in the differential gene list.

3. File Submission

The following files are attached:

- 1_cell_cluster_labels.csv
- 2_differential_expressed_genes.csv
- 3_enrichment_analysis_result.csv

q2

December 8, 2025

```
[6]: import scanpy as sc
import pandas as pd
import gseapy as gp
import numpy as np
import os
import warnings

# Suppress warnings for cleaner output (optional)
warnings.filterwarnings('ignore')

# =====
# Step 0: Load Data
# =====
file_path = '../docs/Gene_expression_mouse_retina.txt'

print(f"Loading data from: {file_path}...")

# Check if file exists
if not os.path.exists(file_path):
    print(f"Error: File {file_path} not found. Please check the path.")
    exit()

try:
    # For .txt files, we usually expect tab delimiters ('\t')
    # .T is used to transpose the matrix (Scanpy expects rows=cells,
    ↪ columns=genes)
    adata = sc.read_csv(file_path, delimiter='\t').T
    print(f"Data loaded successfully. Contains {adata.n_obs} cells and {adata.
    ↪ n_vars} genes.")
except Exception as e:
    print("Failed to read with tab delimiter, trying default comma delimiter...
    ↪", e)
    try:
        adata = sc.read_csv(file_path).T
    except Exception as e2:
        print("Failed to read file. Please check the format.", e2)
        exit()
```

```
# Ensure gene names are unique
adata.var_names_make_unique()
```

Loading data from: ../docs/Gene_expression_mouse_retina.txt...

Data loaded successfully. Contains 3517 cells and 2773 genes.

```
[7]: # =====
# Step 1: Cell Type Clustering
# Methods: Normalization -> Log1p -> PCA -> Neighbors -> Leiden Clustering
# =====
print("Performing normalization and clustering...")

# 1. Normalization and Logarithmize
sc.pp.normalize_total(adata, target_sum=1e4)
sc.pp.log1p(adata)

# 2. Identify Highly Variable Genes (HVG)
sc.pp.highly_variable_genes(adata, min_mean=0.0125, max_mean=3, min_disp=0.5)
adata = adata[:, adata.var.highly_variable]

# 3. Principal Component Analysis (PCA)
sc.pp.pca(adata, svd_solver='arpack')

# 4. Compute Neighbors Graph
sc.pp.neighbors(adata, n_neighbors=10, n_pcs=40)

# 5. Leiden Clustering
# Note: Leiden is the standard community detection algorithm for scRNA-seq
sc.tl.leiden(adata, resolution=0.5, key_added='leiden')

# Save cluster labels
cluster_labels = pd.DataFrame(adata.obs['leiden'])
cluster_labels.columns = ['Cluster_Label']
cluster_labels.to_csv("1_cell_cluster_labels.csv")
print("Step 1 complete: Cluster labels saved to '1_cell_cluster_labels.csv'.")
```

Performing normalization and clustering...

Step 1 complete: Cluster labels saved to '1_cell_cluster_labels.csv'.

```
[8]: # =====
# Step 2: Differential Expression Analysis (DEG)
# Methods: Wilcoxon Rank-Sum Test with Benjamini-Hochberg adjustment
# =====
print("Detecting DEGs for the top two clusters...")

# 1. Summary of cell counts per cluster
cluster_counts = adata.obs['leiden'].value_counts().reset_index()
```

```

cluster_counts.columns = ['Cluster', 'Cell_Count']
cluster_counts.to_csv("2_cluster_summary.csv", index=False)

print("Cluster summary preview:")
print(cluster_counts.head())

# Identify the top 2 clusters
top_2_clusters = cluster_counts['Cluster'].iloc[:2].values
c1, c2 = top_2_clusters[0], top_2_clusters[1]
print(f"Comparing the top 2 clusters: Cluster {c1} vs Cluster {c2}")

# 2. Compute DEGs (Wilcoxon test)
sc.tl.rank_genes_groups(adata, groupby='leiden', groups=[c1], reference=c2,
    ↪method='wilcoxon')
result = sc.get.rank_genes_groups_df(adata, group=c1)

# 3. Filtering
# Thresholds: Adjusted P-value < 0.05 AND Fold Change > 2
alpha = 0.05
fc_threshold = 2
log2fc_threshold = np.log2(fc_threshold)

filtered_deg = result[
    (result['pvals_adj'] < alpha) &
    (abs(result['logfoldchanges']) > log2fc_threshold)
].copy()

# Convert log fold change back to standard fold change for reporting
filtered_deg['fold_change'] = 2 ** filtered_deg['logfoldchanges']

# Save results
output_deg = filtered_deg[['names', 'pvals_adj', 'fold_change',
    ↪'logfoldchanges']]
output_deg.columns = ['Gene', 'P_adj', 'Fold_Change', 'Log2_Fold_Change']
output_deg.to_csv("2_differential_expressed_genes.csv", index=False)
print(f"Step 2 complete: Found {len(output_deg)} DEGs. Saved to_
    ↪'2_differential_expressed_genes.csv'.")

```

Detecting DEGs for the top two clusters...

Cluster summary preview:

	Cluster	Cell_Count
0	0	687
1	1	496
2	2	438
3	3	344
4	4	244

Comparing the top 2 clusters: Cluster 0 vs Cluster 1

Step 2 complete: Found 366 DEGs. Saved to '2_differential_expressed_genes.csv'.

```
[9]: # =====  
# Step 3: Gene Set Enrichment Analysis  
# Methods: Over-Representation Analysis (ORA) via Enrichr API  
# =====  
print("Performing gene set enrichment analysis...")  
  
if len(output_deg) > 0:  
    deg_gene_list = output_deg['Gene'].tolist()  
  
    try:  
        # Using Enrichr API via gseapy  
        # organism='Mouse' is critical for correct gene symbol mapping  
        enr = gp.enrichr(gene_list=deg_gene_list,  
                        gene_sets=['GO_Biological_Process_2021'],  
                        organism='Mouse',  
                        cutoff=0.05)  
  
        enrichment_results = enr.results  
  
        # Sort by Adjusted P-value  
        if 'Adjusted P-value' in enrichment_results.columns:  
            enrichment_results = enrichment_results.sort_values('Adjusted_  
↪P-value')  
  
        enrichment_results.to_csv("3_enrichment_analysis_result.csv")  
  
        if not enrichment_results.empty:  
            top_term = enrichment_results.iloc[0]['Term']  
            print(f"Most relevant biological function: {top_term}")  
        else:  
            print("No significant enrichment found.")  
  
        print("Step 3 complete: Enrichment results saved to_  
↪'3_enrichment_analysis_result.csv'.")  
  
        except Exception as e:  
            print("Enrichment analysis failed (check internet connection or gene_  
↪names):", e)  
    else:  
        print("No DEGs found passing the thresholds, skipping enrichment analysis.")  
  
print("All tasks completed.")
```

Performing gene set enrichment analysis...

Most relevant biological function: visual perception (GO:0007601)

Step 3 complete: Enrichment results saved to '3_enrichment_analysis_result.csv'.

All tasks completed.

[]:

Question 3

We model the normalized expression Y for each gene by the simple linear regression

$$Y = b_0 + b_1x + e,$$

where x is an indicator of condition:

$$x = \begin{cases} 0, & \text{control group C,} \\ 1, & \text{treatment group T.} \end{cases}$$

For a binary predictor with equal numbers of samples in the two groups, b_0 and b_1 have the interpretation

$$b_0 = \text{mean expression in group C}, \quad b_1 = \text{mean}(Y \mid x = 1) - \text{mean}(Y \mid x = 0),$$

i.e. b_1 is the difference between the treatment and control means.

1. Estimation of b_0 and b_1

For each gene we compute

$$\bar{T} = \frac{T_1 + T_2 + T_3}{3}, \quad \bar{C} = \frac{C_1 + C_2 + C_3}{3},$$

then set $b_0 = \bar{C}$ and $b_1 = \bar{T} - \bar{C}$.

As an example, for Gene 1

$$\bar{T}_1 = \frac{4.1 + 5.2 + 2.8}{3} = 4.03\bar{3}, \quad \bar{C}_1 = \frac{5.6 + 3.7 + 3.8}{3} = 4.36\bar{6},$$

so

$$b_{0,1} = 4.36\bar{6}, \quad b_{1,1} = 4.03\bar{3} - 4.36\bar{6} = -0.33\bar{3}.$$

Carrying out the same calculation for all six genes yields

Gene	$\bar{C}$	$\bar{T}$	$b_1 = \bar{T} - \bar{C}$
Gene 1	4.37	4.03	-0.33
Gene 2	4.30	2.73	-1.57
Gene 3	4.50	2.93	-1.57
Gene 4	1.37	2.30	0.93
Gene 5	3.07	2.57	-0.50
Gene 6	2.40	5.43	3.03

(Values are rounded to two decimal places.)

Thus the fitted regression models are

$$Y_g = b_{0,g} + b_{1,g}x + e_g, \quad g = 1, \dots, 6.$$

2. Testing for differential expression

We are told that, over all genes in the genome, the regression coefficients b_1 follow an approximately normal distribution (as illustrated by the given bell curve). We treat the six estimated coefficients $b_{1,g}$ above as a sample from

$$b_1 \sim N(0, \sigma^2).$$

First we estimate σ using the sample standard deviation of the six coefficients:

$$\hat{\sigma} = \sqrt{\frac{1}{n-1} \sum_{g=1}^n (b_{1,g} - \bar{b}_1)^2} \approx 1.75,$$

where $\bar{b}_1$ is the sample mean of the six $b_{1,g}$ (which is essentially 0).

For each gene we then compute the standardized statistic

$$Z_g = \frac{b_{1,g} - \bar{b}_1}{\hat{\sigma}} \approx \frac{b_{1,g}}{1.75}.$$

The resulting Z -scores are

Gene	b_1	$Z_g \approx b_1/1.75$
Gene 1	-0.33	-0.19
Gene 2	-1.57	-0.89
Gene 3	-1.57	-0.89
Gene 4	0.93	0.53
Gene 5	-0.50	-0.29
Gene 6	3.03	1.73

We perform a two-sided test of

$$H_0 : b_1 = 0 \quad \text{vs.} \quad H_A : b_1 \neq 0$$

at significance level $\alpha = 0.10$. For a standard normal distribution,

$$z_{0.95} \approx 1.645,$$

so the rejection region is

$$|Z_g| > 1.645.$$

Comparing the table above with this critical value, only Gene 6 has

$$|Z_6| \approx 1.73 > 1.645,$$

while all other genes satisfy $|Z_g| < 1.645$.

3. Conclusion

At the 10% level of significance, only **Gene 6** shows a regression coefficient b_1 that lies in the outer 10% of the overall b_1 distribution. Therefore,

Gene 6 is the only differentially expressed gene.

All other genes are not significantly differentially expressed between conditions T and C at $\alpha = 0.1$.